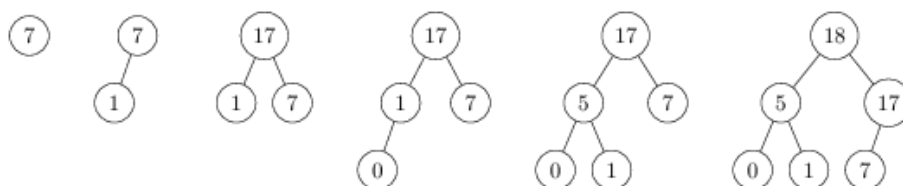


**Domanda B** (7 punti) Dare la definizione di max-heap. Data la sequenza di elementi 7, 1, 17, 0, 5, 18, si specifichi il max-heap ottenuto inserendo, a partire da uno heap vuoto, uno alla volta questi elementi nell'ordine indicato e infine rimuovendo 0. Si descriva sinteticamente come si procede per arrivare al risultato.

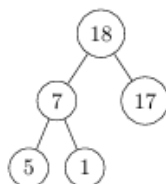
**Soluzione:** Un max-heap è un albero binario ordinato quasi-completo (tutti i livelli completi, a parte l'ultimo, nel quale le foglie devono essere “addossate” a sinistra) con la proprietà che per ogni nodo  $x$ , se  $x$  non è radice, gli antenati di  $x$  hanno chiave maggiore o uguale a quella di  $x$ , o equivalentemente ogni nodo  $x$  ha chiave maggiore o uguale a quella dei suoi discendenti.

Si procede inserendo nel min-heap i vari elementi con la procedura **HeapInsert** a partire da heap vuoto. La procedura inserisce l'elemento come prima foglia utile (ultimo elemento dell'array) e richiama la procedura **MaxHeapifyUp** per ripristinare la proprietà di min-heap (scambiando il nodo con il suo genitore finché la chiave di questo è inferiore a quella del figlio).

Gli inserimenti nell'ordine producono gli heap indicati in figura



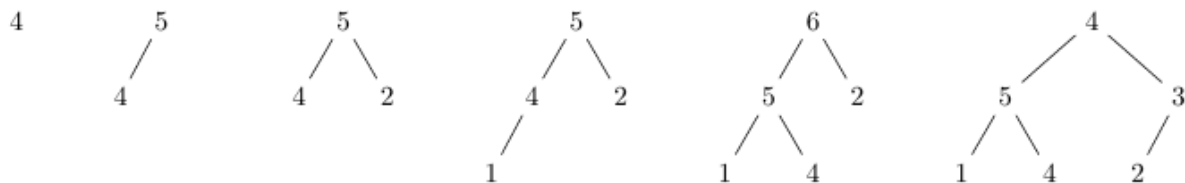
Infine, la rimozione di un nodo con chiave  $x$  si realizza sostituendolo con l'ultima foglia  $y$  e richiama **MaxHeapifyUp**, se  $y > x$  oppure **MaxHeapify**, se  $y < x$ . Nello specifico per eliminare 0 lo si rimpiazza con l'ultima foglia 7 e si richiama **Max**, ottenendo il min-heap



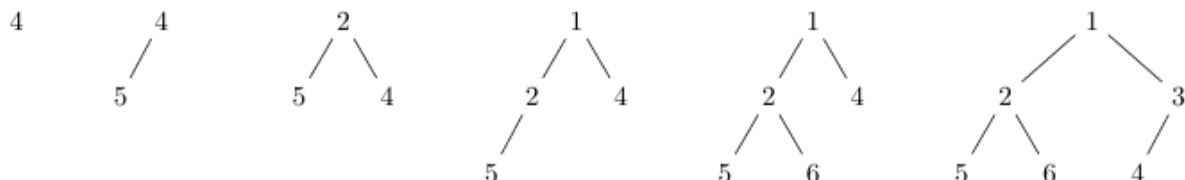
che in forma di array è [12, 19, 15, 20, 21, 22].

**Domanda B** (5 punti) Si dia la definizione della struttura dati max-heap. Si supponga di inserire in un max-heap inizialmente vuoto, in successione, gli elementi 4, 5, 2, 1, 6, 3. Fornire la rappresentazione (ad albero) del max-heap così ottenuto. Si motivi la risposta mostrando la sequenza di max-heap ottenuti.

**Soluzione:** Per la definizione di max-heap e la procedura di inserzione di un elemento si rimanda al testo. La sequenza di max-heap ottenuta con l'inserimento degli elementi nell'ordine indicato è:



mentre invece la sequenza di min-heap è:



**Esercizio 1** (9 punti) Realizzare una funzione `intersect(A1,A2,n)` che dati due array di interi `A1` e `A2`, organizzati a min-heap, con capacità `n`, restituisce un nuovo array `A`, ancora organizzato a min-heap, che contiene l'intersezione dei valori contenuti in `A1` e `A2`. Nel caso gli array contengano più occorrenze dello stesso valore  $v$ , l'intersezione mantiene il numero minimo di occorrenze di  $v$  (ad es. se `A1` contiene i valori 1,2,2,2 e `A2` contiene i valori 1,1,2,2 allora `A` conterrà 1,2,2). Valutarne la complessità.

**Soluzione:** Per costruire l'intersezione, si procede nel modo seguente. Posso estrarre gli elementi di ciascun min-heap, in ordine crescente, semplicemente con una sequenza di estrazioni di minimo, ciascuna delle quali ha costo logaritmico. Se da `A1` estraggo  $v_1$  e da `A2` estraggo  $v_2$ , se  $v_1 = v_2$ , inserisco il valore comune nell'intersezione. Se invece  $v_1 < v_2$ , certamente  $v_1$  non è nell'intersezione, dato che tutti gli elementi rimanenti in `A2` sono  $\geq v_2$ , quindi posso scartare  $v_1$ , estrarre un nuovo elemento da `A1` e continuare. Se  $v_1 > v_2$  procedo dualmente.

In questo modo ottengo gli elementi dell'intersezione in ordine crescente. L'ultima osservazione è che posso inserirli in questo modo in `A`, che risulterà un array crescente, che è un caso particolare di min-heap.

Lo pseudocodice può essere il seguente:

```

intersect(A1,A2,n)
  allocate A[1..n]
  i=0                                // ultimo elemento occupato in A
  while (A1.heapsize > 0) and (A2.heapsize > 0)
    v1 = Min(A1)
    v2 = Min(A2)
    if (v1 = v2)
      i++
      A[i]=1
      ExtractMin(A1)
      ExtractMin(A2)
    elseif (v1 < v2)
      ExtractMin(A1)
    else
      ExtractMin(A2)

  A.heapsize=i
  return A

```

La complessità di ogni `ExtractMin` è  $O(\log n)$  mentre `Min` ha costo costante. Il numero di estrazioni è chiaramente limitato da  $2n$  e questo porta a dedurre che il costo complessivo è  $O(n \log n)$ .